Danmarks Tekniske Universitet

Telecommunications project with Arduino (34338)

Fire detector

Elias Valtteri Vähälummukka
s234379

Jacob Osvald Beeken Holm
s234413

Marcus Dystrup Pedersen
s234392

Mathias Evendorf Højsholt Schyth
s235476

January 25, 2026

Contents

1 Introduction **2**

2 Project Usage **2**

3 Finite State Machine (FSM) **3**
3.1 Fire Node . . . 3
3.2 Fire Main . . . 4

4 System Setup **4**
4.1 Fire Main . . . 4
4.2 Fire Nodes . . . 5
4.3 System components . . . 5
4.4 How to run the code . . . 6
4.5 ThingSpeak . . . 6
4.6 TalkBack . . . 8

5 Challenges **9**
5.1 ThingSpeak . . . 9
5.2 Analog sensor implementation . . . 10
5.3 Human presence sensor (libraries) . . . 10
5.4 Timing . . . 10
5.5 Reduction of ESP32 resource usage . . . 11

6 Discussion **11**

7 Conclusion **12**

8 Distribution **13**
8.1 Coding distribution . . . 13
8.2 Report distribution . . . 13

9 Workflow **14**

10 Appendix **15**

1 Introduction

This project aims to design and implement an early risk fire detector with cloud logging and remote control. The project is based on ESP32 micro controllers with WiFi connectivity which allows network communication for multiple modules.

The system continuously monitors air quality using gas/smoke and environmental sensors and detects potentially dangerous situations using threshold- and rate-of-change-based logic. The system consists of two sensor nodes and one main control node:

The sensor node ESP32's are placed in different rooms throughout the building and will measure co2/gas concentration, temperature and human presence. The main ESP32 is equipped with a screen and acts as a local alarm panel, displaying system status and identifying which node has triggered an alarm.

When abnormal conditions are detected, the affected sensor node enters ALARM state, activates local feedback (LED and buzzer), and sends the event to a cloud backend. The server then broadcasts an ALARM command to all ESP32 devices, ensuring synchronized system behavior across all nodes.

The main ESP32 allows users to:

- See which node triggered the alarm
- Mute alarms system-wide
- Reset the system after inspection

A web-based backend enables live monitoring of sensor data, historical logging, and remote interaction via downlink commands (e.g. mute alarm and reset system).

2 Project Usage

The developed system is intended as an early-warning fire risk detection and monitoring solution in indoor environments, such as residential buildings, offices, laboratories, or small industrial facilities. By deploying multiple sensor nodes in different rooms, the system provides distributed monitoring of environmental conditions that may indicate a fire or hazardous air quality event.

During normal operation, each sensor node continuously measures gas and smoke concentration, temperature, and human presence. These measurements are transmitted over a wireless network to the cloud backend and, when relevant, to the main control node. The system operates autonomously, requiring no user intervention.

When a sensor node detects values exceeding thresholds or observes a rapid change in sensor readings, it transitions into ALARM state. In this case, local visual and audible indicators are activated to alert occupants in the immediate area. Simultaneously, the event is logged in the cloud backend and sent to the main control node, which displays detailed information about the alarm, including the originating sensor node.

The main control node serves as the user interface for on-site interaction. Users can identify which room or sensor node triggered the alarm, mute audible alarms across all nodes to, and reset the system once the situation has been assessed and resolved. These controls allow for coordinated and controlled handling of alarm events without requiring physical access to each sensor node.

In addition to local interaction, the web-based backend enables remote usage of the system. Authorized users can monitor live sensor data, review historical logs for analysis and reporting, and issue downlink commands such as muting alarms or resetting the system. This functionality makes the system suitable for both real-time supervision and long-term environmental monitoring, improving situational awareness, and improving overall fire safety management.

3 Finite State Machine (FSM)

The main Fire node and its Sensor nodes are all designed around a FSM. This is to make sure we always change the State of the nodes while keeping server data up to date, even updating without a server connection. We made sure that the highest priority is the Firenodes own data, making sure it ALARMS and then checks server.

3.1 Fire Node

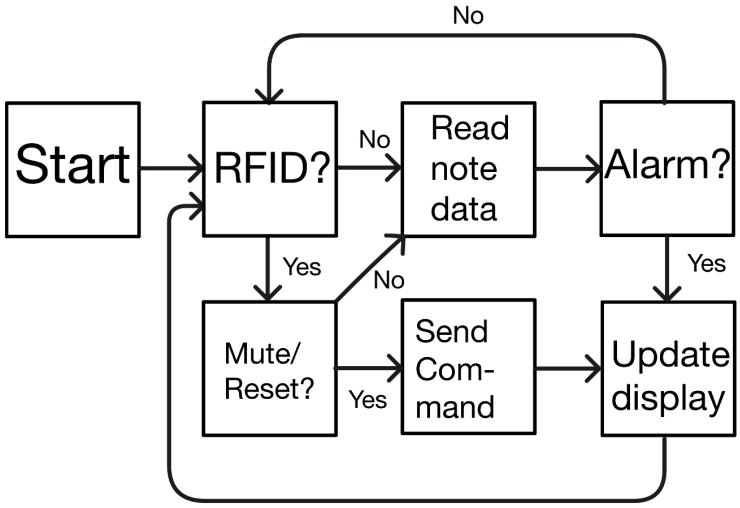


Figure 1: Finite state machine

The nodes continuously read data and check if any of the node sensors read an extraordinary event, and if it does, it updates the state and checks for a server connection. If it finds a connection, it uploads the data, checks if the old data are different from the new data, and if they are, it determines what state we should assign it to, and updates the state.

3.2 Fire Main

The main Fire node operates as the physical layer for the system and Firenode Data. It continuously receives data from the server and displays the system state, established connections, and the trigger if the system enters an ALARM state. We use a simple FSM to incorporate our different GUIs on the Main node while checking for RFID authentication and being able to MUTE or RESET the nodes by sending commands to the server.

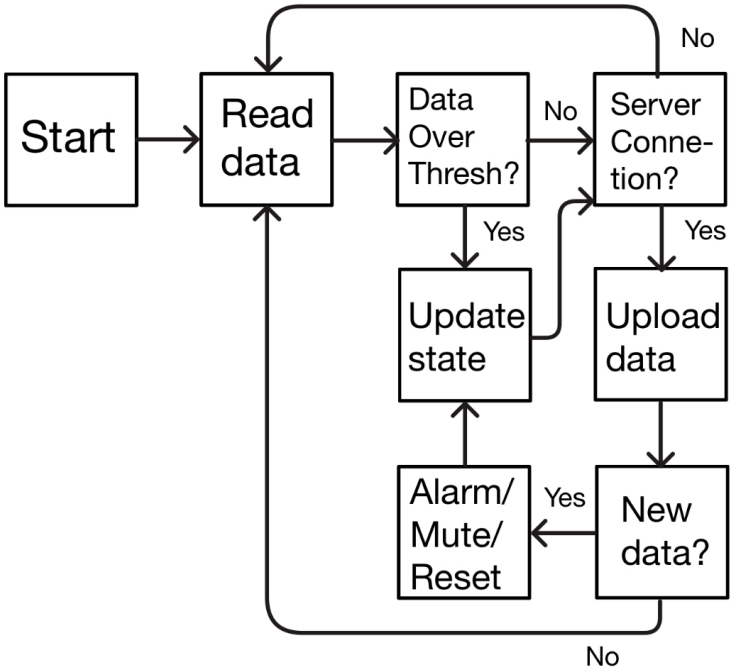


Figure 2: Finite state machine

4 System Setup

The system consists of two different types of nodes, a central main node and multiple firenodes responsible for data collection. The nodes are built around the ESP32 nodemcu. And communicate to eachother using ThingSpeak.

4.1 Fire Main

The Fire Main is equipped with an RFID scanner used to reset or mute the alarm once it has been triggered. This is handled through two different keys, a Mute key and a Reset key. These keys are intended to be issued to the fire department, or alternatively, one of their existing keys can be registered in the system. This ensures that firefighters or an admin can reset the alarm in the case of a false trigger, or mute it so they can focus while handling an active fire.

The main hub features two different screens: a normal-operation screen and an alarm screen.

Each screen displays information relevant to the current system state. The screen will show:

System info shown on normal state

- Server and Fire Node connection
- Amount of fire nodes connected
- WiFi connection strength
- Time since last data was received

System info shown on Alarm state

- Server and Fire Node connection
- Node that triggered
- Type of trigger
- Presence of humans

(a) NORMAL Screen

(b) ALARM Screen

Figure 3: Oled Screens

4.2 Fire Nodes

As for the firenodes, these are equipped with temperature, gas, infrared and motion sensors used to measure room conditions. To alert the user in case of fire a led will indicate the system state and a buzzer will make noise to catch the users attention. To connect all of the nodes, ThingSpeak server is implemented. The firenodes send the gathered data to ThingSpeak while the main node reads from the ThingSpeak server and issues commands to the firenodes, to control the alarm state. All of the gathered data is visible on a ThingSpeak dashboard.

4.3 System components

Fire Main

- ESP32 development board (with WiFi)
- RFID reader (RFID-RC522)
- OLED screen (SSD1306)

One Sensor Node

- ESP32 development board (with WiFi)
- Gas/temperature sensor (SCD30 or ENS160/AHT2x)
- Human presence sensor (HLK-LD2450)
- 2 buzzers
- RGB LED

Supporting hardware

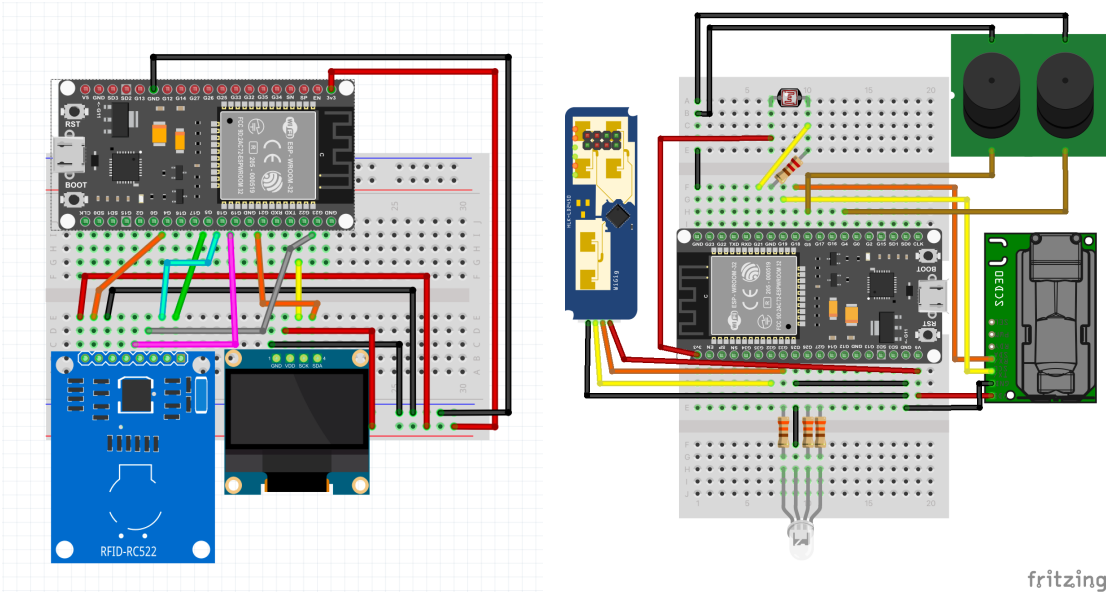
- Breadboard
- Wires
- Resistors (330 Ω)

Software

- ThingSpeak
- Arduino IDE

A wiring diagram for the Main Hub and the FireNodes can be seen below. The key point is that the FireNodes allow their sensors, buzzers, and LEDs to be exchanged with only minor changes to the sensor libraries, as long as the libraries match the connected hardware. This enables a

wide range of sensors and sensor layers to be used on the same node, ensuring compatibility across many different environments. This flexibility is demonstrated by our use of two different gas and temperature sensors on FireNode 1 and FireNode 2, highlighting the adaptability of our sensor platform.



(a) Fire Main (b) Fire Node

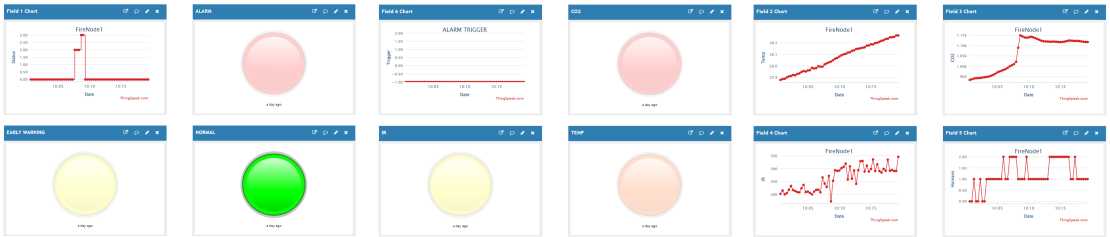
Figure 4: System schematic

4.4 How to run the code

The code is simple to run, as you only need to upload the Fire Main and the FireNodeOne/Two to a system as shown above, and supply it with power. By importing the repository from GIT, you have access to the custom libraries enabling the code to run correctly. Wifi and a Thingspeak channel is needed for optimal function, SSID, Password, channel ID and API key can be uploaded to the code before flashing.

4.5 ThingSpeak

ThingSpeak is used as the cloud platform for collecting and visualizing data from the fire nodes. Each fire node uploads the following parameters: system status, fire trigger state, temperature, CO_2 concentration, IR light level, and human presence. ThingSpeak provides live plots of the sensor values, while visualization widgets are used to clearly display the binary trigger and status states in fig 5.



(a) Status (b) Trigger (c) Raw Sensor Data

Figure 5: ThingSpeak Website Visualization

Each ThingSpeak channel supports a maximum of eight data fields. Since each fire node requires six fields, a dedicated channel is used for each node. Additionally, ThingSpeak enforces a daily upload limit of 86 400 messages per channel, corresponding to 259 200 individual field values per day. To remain within this limit, data from each fire node is uploaded at fixed intervals rather than continuously.

In order to reduce network traffic and comply with the upload restrictions, sensor values are sampled continuously and accumulated locally on the ESP32. Every 20 seconds, the accumulated samples are averaged and transmitted as a single data point per field. This results in an effective update rate of one data point every 20 seconds, which explains the observable delay in the live graphs.

The upload logic is implemented as shown in Listing 1. A non-blocking timing mechanism based on `millis()` ensures that uploads occur periodically without halting the main program execution. Before transmission, temperature and CO_2 values are averaged to reduce noise and avoid unnecessary data fluctuations.

Listing 1: ThingSpeak upload with averaged sensor values

```
const unsigned long POST_INTERVAL_MS = 20000; // ThingSpeak upload interval
// ---------- THINGSPEAK UPLOAD ----------
if (now - lastPostMs ≥ POST_INTERVAL_MS) {
    lastPostMs = now;

    // Average upload values if samples are available
    if (accCnt > 0) {
        s1.T    = accT / accCnt;
        s1.eCO2 = accCO2 / accCnt;
    }

    const int httpCode = ts.upload(s1);
    Serial.println(httpCode == 200 ?
                   ThingSpeak OK  :  ThingSpeak FAIL );

    // Reset accumulators
    accT = 0;
    accCO2 = 0;
    accCnt = 0;
```

```
}
```

This approach ensures reliable data transmission while minimizing bandwidth usage and staying within the limitations imposed by the ThingSpeak platform.

4.6 TalkBack

ThingSpeak TalkBack is used as a centralized command distribution mechanism to ensure that all FireNodes operate in a consistent system state. This is particularly important in fire scenarios, where all nodes must enter an alarm state even if only a single FireNode has detected a fire.

The synchronization logic is handled in the cloud layer using React on incoming data streams. When the status field of any FireNode transitions to the *alarm state* (`state = 2`), a ThingHTTP request is triggered. This request issues an API call to the corresponding TalkBack channel, enqueueing an `ALARM` command. Each FireNode is subscribed to its own TalkBack channel and periodically polls for new commands using a non-blocking `millis()`-based timer with a polling interval of five seconds.

All system states received through TalkBack are latched locally on the FireNode, with the exception of the normal state. This ensures that alarm and mute states persist regardless of subsequent sensor readings. As a result, false resets are avoided, and alarm conditions remain active until an explicit `RESET` or `MUTE` command is received. This design guarantees correct system behavior across node resets and temporary communication interruptions.

Reset and mute operations are initiated through the central Fire Hub using RFID authentication. When a valid RFID key is detected, the Fire Hub issues API calls that first clear all queued commands in the TalkBack channels and then insert a single, explicit command corresponding to either `RESET` or `MUTE`. By removing stale commands before issuing a new one, each FireNode is guaranteed to observe only the current, intended system state.

To ensure consistent interpretation of commands, a custom TalkBack library is implemented. The library maps textual TalkBack commands to internal integer codes, as shown in Listing 2.

Listing 2: Registration of supported TalkBack commands

```
tb.addCommand( NORMAL , 0);
tb.addCommand( ALARM ,  1);
tb.addCommand( RESET ,  2);
tb.addCommand( MUTE ,   3);
```

Each FireNode periodically polls its TalkBack channel using the function shown in Listing 3. If no commands are present, the function returns `-1`, and execution continues without modifying the system state. This ensures a fully non-blocking main loop and minimizes unnecessary processing.

Listing 3: Non-blocking TalkBack polling and command handling

```
const unsigned long TALKBACK_INTERVAL_MS = 5000;
// ---------- TALKBACK POLL ----------
if (now - lastTalkbackMs ≥ TALKBACK_INTERVAL_MS) {
    lastTalkbackMs = now;
    const int code = tb.poll();
    if (code ≥ 0) {
    handleTalkBackCode(code);
    }
}
```

By combining cloud-based command orchestration with local state latching and non-blocking polling, the TalkBack system provides a robust and scalable method for maintaining consistent behavior across all FireNodes.

5 Challenges

The main challenges we faced during this project were:

- ThingSpeak
- Analog sensor implementation
- Human presence sensor (libraries)
- Timing
- Reduction of ESP32 resource usage

5.1 ThingSpeak

During the initial project planning phase, ThingSpeak was selected as the primary communication platform. However, as the system architecture evolved and the need for reliable communication between multiple FireNodes became clear, several limitations of ThingSpeak emerged. ThingSpeak is primarily designed for data logging and analytical visualization, and it is inherently optimized for one-to-one communication rather than coordinated interaction between multiple nodes.

Because ThingSpeak is intended for data visualization rather than distributed node communication, additional services such as multiple channels, React, ThingHTTP, and TalkBack were required to achieve functionality comparable to a message-oriented system. In a system based on MQTT and Node.js, a publish–subscribe mechanism could have been used to distribute commands efficiently to all nodes, while data visualization could have been implemented using a lightweight front-end solution based on HTML and CSS.

As a result, ThingSpeak is not an ideal platform for this type of distributed control system. Nevertheless, through careful system design, non-blocking timing, and the use of chained API calls, it was possible to implement a functional command distribution mechanism. This approach

increased system complexity and introduced additional latency compared to a dedicated MQTT-based solution, but it ultimately allowed the required functionality to be achieved within the constraints of the chosen platform.

5.2 Analog sensor implementation

As our project initially did not include any analog sensors, this part of the implementation proved to be challenging. By incorporating a photosensitive resistor (LDR), we were able to successfully integrate and utilize an analog sensor in the system.
When the LDR is initialized, it measures the surrounding light and saves it as the normal. It then uses `float instSlope = dIr / dtMs;` to check how much the light has changed and how quickly it changed, this gives a speed of change and not just brightness. To detect big jumps(Light gets turned on) it uses a bool to check if the light jumps a lot in a very short time `bool isStep ... = (fabsf(dIr)≥P.stepJumpAbs)&& (dtMs_u ≤250);` To make sure we still know what normal light looks like we change our baseline with small smooth increments even if lights have been turned on and off `baseline = baseline + P.baselineAlpha * clampedRise;`. It smooths the slope with a simple "smoother": `slope = slope + P.slopeAlpha * (instSlope - slope);`. When it has done every calculation it then checks for a fire using `bool looksFireLike = (!out.stepChange)&& ... (out.rise ≥P.minRiseAbs)&& (out.slope ≥P.minSlope);`, if it looks like a fire, it makes sure its consistent over several seconds and not a fluke in the system `out.fireLikely = (confirmStartMs ... != 0)&& ((nowMs - confirmStartMs)≥P.confirmMs);`. This approach was chosen to avoid false triggers caused by sudden changes, such as turning on the lights.

5.3 Human presence sensor (libraries)

The human presence detector presented challenges primarily related to library compatibility and configuration, which required careful testing and adjustment to ensure reliable operation. This ended up taking more time than accounted for, and halted the integration process by a considerable amount.

5.4 Timing

One of the major challenges in this project was achieving correct timing. The system consists of multiple nodes, each running different tasks at unique intervals. In particular, timing issues arose with the LED and buzzer, as they needed to blink or beep continuously while the system was in the Muted or Danger state. Since sensor data was only read every two seconds, the LED and buzzer were also updated at this interval, resulting in behavior that was noticeably too slow.
Additional timing challenges occurred during debugging, as well as when uploading and downloading data from the server.

5.5 Reduction of ESP32 resource usage

Optimizing resource usage on the ESP32 was a significant challenge, as both memory and processing power had to be carefully managed to ensure stable operation of all system functions. During development, program memory usage reached a critical level of approximately 97%, making efficient use of available resources essential.

To reduce computational load during runtime, a bitmap-based approach was implemented for the OLED display. Instead of recalculating text and graphical elements on every screen update, predefined bitmaps are rendered directly. Although this approach increases program memory usage, it significantly lowers processing overhead during normal operation.

The resulting imbalance between program memory usage (approximately 97%) and dynamic memory usage (approximately 13%) was considered acceptable and partly intentional. Display layouts and graphical assets are stored in non-volatile memory, while volatile memory usage is kept low by limiting persistent data storage. Sensor data is therefore not retained long-term on the ESP32; only approximately 20 seconds of data is buffered locally before being transmitted to the server and subsequently cleared. This strategy minimizes dynamic memory usage and reduces the risk of memory-related faults.

In addition, system functions were optimized to avoid unnecessary execution. Boolean return values are used to indicate successful execution, error states, or whether further actions are required. This ensures that functions are only executed when all prerequisites are met, preventing redundant processing and avoiding situations where fire nodes could enter infinite loops due to missing dependencies or invalid inputs.

6 Discussion

The results of this work highlight several important considerations regarding system architecture, security, reliability, and resource efficiency. One of the most significant findings is the limitation of using ThingSpeak as a backend platform for node control. While ThingSpeak is convenient for data visualization and basic IoT prototyping, it is not well suited for real-time control or bidirectional communication with distributed nodes. This limitation motivates the adoption of a more capable backend solution that supports lower latency and finer control logic. A dedicated backend would allow for improved responsiveness and better integration of control and monitoring functionalities.

Security is another critical aspect addressed in this system. The use of encrypted RFID cards, along with encrypted server communication, significantly enhances protection against unauthorized access and data interception. Without encryption, RFID systems are vulnerable to cloning and replay attacks, and unprotected network traffic can be easily intercepted. By implementing encryption at both the device and server levels, the system ensures confidentiality, integrity, and authentication, making it more suitable for real-world deployment where security requirements are higher.

System reliability is further improved by introducing a backup mechanism for wireless data com-

munication. Relying solely on WiFi creates a single point of failure, particularly in environments where network availability may be unstable. By incorporating an alternative communication method, the system can maintain functionality even when WiFi becomes unavailable. This redundancy increases robustness and ensures continued operation, which is especially important for access control or monitoring applications where downtime is unacceptable.

Finally, reducing ESP32 resource usage plays an essential role in improving overall system performance. Optimizing memory consumption, processor load, and network overhead allows the microcontroller to operate more efficiently and reliably. Efficient resource management therefore contributes directly to system stability, responsiveness, and future extensibility.

7 Conclusion

This project successfully designed and implemented a distributed early fire risk detection system using ESP32 micro controllers with wireless communication and cloud integration. By combining multiple sensor nodes with a centralized main control node, the system enables real time monitoring of environmental conditions such as gas concentration, temperature, and human presence to detect potentially hazardous situations at an early stage.

The use of finite state machines across both fire nodes and the main hub ensured structured behavior, reliable state transitions, and continued operation even during network interruptions by prioritizing local alarm handling. Cloud connectivity enabled centralized monitoring, historical data logging, and remote control, while encrypted RFID authentication and secure server communication improved overall system security.

Despite challenges related to ThingSpeak's limitations, timing constraints, and ESP32 resource usage, the system achieved stable performance through optimization and careful architectural choices. In general, the project demonstrates a scalable and modular IoT-based fire detection solution using low-cost hardware. While further improvements could increase reliability, the implemented system fulfills its purpose as an effective early-warning fire risk detector and provides a strong foundation for future development.

8 Distribution

Distribution of Responsiblity

Throughout this project we have been helping each other along, and all of us have been part of each implementation. We have all been writing the report and integrating the code. However, each of us have been assigned implementations to be main responsible. The distribution of implementations can be seen as below:

8.1 Coding distribution

Task	Jacob	Elias	Mathias	Marcus
Sensor libraries and drivers	20%	0%	0%	80%
Sensor-node interfacing	50%	0%	50%	0%
RFID library and integration	0%	45%	10%	45%
State-machine logic	0%	25%	55%	20%
Alarm, buzzer and LED logic	0%	0%	100%	0%
Backend and cloud integration	0%	10%	0%	90%
OLED UI and display handling	0%	0%	0%	100%
Firmware integration (FireNodes)	10%	10%	40%	40%
Firmware integration (MainNodes)	0%	0%	0%	100%
Documentation and Doxygen	0%	0%	0%	100%
Development environment setup	0%	0%	0%	100%

8.2 Report distribution

Task	Jacob	Elias	Mathias	Marcus
Introduction	0%	0%	100%	0%
Project Usage	100%	0%	0%	0%
Finite State Machine (FSM)	0%	0%	0%	100%
System Setup	0%	25%	25%	50%
Challenges	0%	0%	50%	50%
Discussion	100%	0%	0%	0%
Conclusion	100%	0%	0%	0%
Fritzing diagrams	100%	0%	0%	0%
Flow Charts	50%	0%	0%	50%

9 Workflow

```
* f688111 | 2026-01-20 | Marcus Dystrup Pedersen | Merge branch 'main' of ...
    gitlab.gbar.dtu.dk:s234392/telecom_projekt_fire_prevention
|\
| * d2c93af | 2026-01-20 | Marcus Dystrup Pedersen | Edit README.md
* | ed81025 | 2026-01-20 | Marcus Dystrup Pedersen | Removed API Keys, SSID and PSWD from ...
    all code and library examples and fixed some doxygen problems
|/
* 74878f9 | 2026-01-20 | Marcus Dystrup Pedersen | Made Doxyfile, Doxygen comments in all ...
    of main code and some libraries
* efa6088 | 2026-01-20 | Marcus Dystrup Pedersen | Made Doxyfile, Doxygen comments in all ...
    of main code and some libraries
|\
| * dbc61fc | 2026-01-20 | Marcus Dystrup Pedersen | Edit README.md
| * 23bcadb | 2026-01-20 | Marcus Dystrup Pedersen | Edit README.md
* | f9bdc91 | 2026-01-20 | Marcus Dystrup Pedersen | Added Doxygen for all libraries, ...
    added Latex in Doxygen and ra Doxygen
|/
* 7ca8de6 | 2026-01-20 | Marcus Dystrup Pedersen | Made Doxyfile, Doxygen comments in all ...
    of main code and some libraries
* 7b28b31 | 2026-01-20 | Mathias Schyth | Deleted gitignore things
* 89873b9 | 2026-01-19 | Marcus Dystrup Pedersen | Fixed gitignore
* 105e82b | 2026-01-19 | Marcus Dystrup Pedersen | Edit .gitignore
* 29b64c6 | 2026-01-19 | Marcus Dystrup Pedersen | Fixed folder structure and added platformio
* 586c0a9 | 2026-01-19 | Marcus Dystrup Pedersen | Fixed code Snippets, Changed Pin ...
    possitions Finished FireNodeOne code
* 0391e6b | 2026-01-19 | Elias Vhlummukka | Added thingspeak pipeline
* 1da6b0e | 2026-01-19 | Jacob Holm | Fritzing files for report
* da55f7a | 2026-01-19 | Marcus Dystrup Pedersen | Added LDR Lib and patched it into ...
    sensor node two code
* 089ba41 | 2026-01-17 | Marcus Dystrup Pedersen | Made Full code for Firenode2, it now ...
    works with server and new sensor
* 469bd89 | 2026-01-16 | Mathias Schyth | Added node 1
* 1f0ec68 | 2026-01-16 | Marcus Dystrup Pedersen | Added Main Firewall code
* 8aef221 | 2026-01-16 | Elias Vhlummukka | added rfid reader code
* 4e9d70e | 2026-01-15 | Mathias Schyth | updated alarmio with status switchcase
* 4f0715b | 2026-01-15 | Marcus Dystrup Pedersen | Added Multiple ThinSPeak ...
    functionallities, Upload, Download, Read Talkback. Corresponding examples and ...
    libraries have been made
* 8d71f31 | 2026-01-15 | Marcus Dystrup Pedersen | Human Tracker Library added
* 5810f5e | 2026-01-15 | Jacob Holm | fixed alarmio library to a switchcase with states
* 5dbdb89 | 2026-01-14 | Elias Vhlummukka | Test snippet for things-peak
* 3297e43 | 2026-01-14 | Marcus Dystrup Pedersen | Removed File extensions, Added screen ...
    h file
* d6aac81 | 2026-01-14 | Elias Vhlummukka | Gitignored macbook tomfoolery
* 7ca7b74 | 2026-01-14 | Jacob Holm | Sensor headers added
* b892b02 | 2026-01-14 | Mathias Schyth | Final alarmio update hopefully
* 006b4b2 | 2026-01-14 | Mathias Schyth | might work now actuator thingy
* 7846fa6 | 2026-01-14 | Mathias Schyth | fixed actuator example
* e63ad79 | 2026-01-14 | Mathias Schyth | Made the actuators a library
* db9255d | 2026-01-14 | Marcus Dystrup Pedersen | Added WifiHelper Library Marcus
* 98ffd3c | 2026-01-14 | Mathias Schyth | Updated actuator
```

```
* b688b96 | 2026-01-14 | Mathias Schyth | Added actuators-file
* e8c7f65 | 2026-01-13 | Elias Vhlummukka | added project plan and group contract
* 94c65d7 | 2026-01-13 | Marcus Dystrup Pedersen | Folder structure for projekt updated
* 7abec33 | 2026-01-13 | Marcus Dystrup Pedersen | Folder structure for projekt has been made
* 55c7006 | 2026-01-13 | Elias Vhlummukka | Added gitignore and nuked templaet readme
* edb3eed | 2026-01-13 | Marcus Dystrup Pedersen | Initial commit
```

10 Appendix

Code can be found on github: Github Code